

Vibe Coding Prozess

Hier dokumentieren wir, wie wir tatsächlich gearbeitet haben. Uns war von Anfang an wichtig, nicht einfach wild drauflos zu prompten, sondern einen festen Ablauf zu haben. Wir wollten verstehen, was entsteht, und die KI als Werkzeug einsetzen und nicht als Ersatz fürs eigene Denken. Entstanden sind dabei zwei Repositories mit zusammen über hundert Commits, der größte Teil davon im Frontend.

Von der Idee zur Architektur

Wir haben mit einer Idee gestartet und nicht mit einem fertigen Lastenheft. Bevor wir die erste Zeile Code erzeugt haben, haben wir überlegt, aus welchen Bausteinen die App besteht und wie sie zusammenhängen. Daraus ergab sich früh die Aufteilung in ein Frontend und ein Backend mit eigener Datenbank.

Besonders geholfen hat uns, die Produktidee einmal sauber festzuhalten. Unser roter Faden ist ein durchgehender Lernworkflow vom Kurs über die Vorlesung und das Dokument bis zu Zusammenfassung, Karteikarten und Prüfungsvorbereitung. Diese Linie haben wir bewusst dokumentiert, damit die App nicht zu einem beliebigen PDF-Chatbot oder einer Sammlung einzelner KI-Spielereien wird. Genau das war unsere wichtigste Leitplanke bei jeder Entscheidung.

Unser Setup und die Tools

Als Haupt-Werkzeug haben wir Claude Code eingesetzt, also einen KI-Agenten direkt in der Entwicklungsumgebung. Ergänzend haben wir ChatGPT genutzt, vor allem zum Brainstormen von Ideen, für Erklärungen und als zweite Meinung. Wenn wir unsicher waren, ob eine Lösung von Claude Code sauber ist, haben wir sie von ChatGPT gegenchecken lassen. Genau das deckt sich mit dem Hinweis aus der Vorlesung, eine andere KI in Maßen zum Prüfen zu verwenden.

Damit die KI nicht bei jeder Aufgabe von vorne raten muss, haben wir ihr im Frontend-Repository festen Kontext mitgegeben:

- **AGENTS.md** hält die dauerhaften Produkt- und Projektregeln fest, zum Beispiel die Produktvision und die Design- und Sicherheitsvorgaben.
- **CLAUDE.md** beschreibt, wie der Agent arbeiten soll, etwa dass er bestehenden Code vor Änderungen zunächst prüft.
- **CURRENT_STATE.md** wurde zu Beginn als zusätzliche Momentaufnahme des Umsetzungsstands gepflegt. Später wurde die Datei nicht mehr laufend aktualisiert und dient deshalb nur noch als historische Orientierung.

Technisch steht das Frontend auf Next.js mit React und TypeScript, das Backend läuft über Supabase mit Postgres, Migrationen und serverseitigen Funktionen. Für die Suche in den Unterlagen haben wir zunächst eine Vektorsuche eingesetzt und später um eine hybride Suche erweitert, die semantische

und klassische Suchverfahren kombiniert. So kann der Assistent Fragen mit Bezug zum echten Kursmaterial beantworten.

Wie wir mit der KI gearbeitet haben

Unser Ablauf war fast immer gleich und hat sich über das Projekt eingespielt:

- **Kleine, klare Aufgaben statt riesiger Prompts.** Wir haben jeweils ein Feature mit klaren Akzeptanzkriterien beschrieben, nach dem Muster baue X, das Y leistet. Große Würfe am Stück haben bei uns selten funktioniert.
- **Erst denken, dann umsetzen lassen.** Wir haben vorgegeben, was passieren soll, und die KI hat es umgesetzt. Die Richtung kam immer von uns.
- **Prüfen, bevor etwas gebaut wird.** Wir haben die KI angehalten, den vorhandenen Code erst anzuschauen. So ist weniger doppelter oder widersprüchlicher Code entstanden.
- **Iterieren in engen Schritten.** Statt eine ganze Anwendung auf einmal zu ändern, haben wir einzelne Teile nachgeschärft und zwischendurch aufgeräumt.

Diese Arbeitsweise spiegelt sich auch in der Struktur wider. Wir haben in klar benannten Feature-Zweigen gearbeitet, zum Beispiel für die Anbindung des KI-Assistenten, die Zusammenfassungen, das Design oder die Bezahlseite.

Testen und Qualität

Testen war für uns kein nachträglicher Schritt, auch wenn wir das erst mit der Zeit richtig ernst genommen haben. Für die Datenbank haben wir eigene Tests geschrieben, dazu kommen Unit-Tests und End-to-End-Tests für die wichtigsten Abläufe. Die wichtigsten Prüfungen laufen zusätzlich automatisiert über die CI, insbesondere bei den dafür konfigurierten Pull Requests und Pushes. Dort werden der Stil, die Tests und ein frischer Build geprüft. So fallen Fehler früh auf und nicht erst in der Präsentation.

Zusammenarbeit im Team

Wir haben im Team über Pull Requests gearbeitet. Features und größere Änderungen liefen in der Regel über eigene Zweige und Pull Requests und wurden vor dem Zusammenführen geprüft. Der grobe Weg war Feature-Zweig, dann Pull Request, dann automatische Prüfung durch die CI und erst danach das Zusammenführen. Staging und Produktion sind als getrennte Umgebungen angelegt, damit Änderungen zuerst auf Staging geprüft werden können. Dadurch konnten mehrere von uns parallel arbeiten, ohne sich gegenseitig den Stand kaputt zu machen.

Dokumentation

Parallel zum Code haben wir viel dokumentiert, bewusst von uns selbst geschrieben und nicht von der KI. In eigenen Dokumenten haben wir unter anderem die Vektorsuche, den Datei-Speicher, die Anmeldung, das Datenmodell und die Verarbeitung der Dokumente beschrieben. Das hat zwei

Gründe. Erstens zwingt es uns, den Code wirklich zu verstehen. Zweitens finden wir uns dadurch im Projekt schneller zurecht, wenn eine Funktion später angepasst werden muss.

Deployment

Das Backend wird über die CI ausgerollt. Migrationen und Funktionen gehen zuerst auf eine Staging-Umgebung, auf der wir kritische Abläufe wie Anmeldung und Datenzugriffe prüfen. Erst danach ist der Stand für die Produktion vorgesehen. Dieser getrennte Weg hat uns geholfen, Änderungen in Ruhe zu testen, bevor sie für alle sichtbar werden.

Fazit zum Prozess

Am meisten gebracht hat uns die Disziplin, vor dem Prompten zu planen und nach dem Prompten zu prüfen. Vibe Coding war dadurch nicht weniger Arbeit, aber eine andere Art von Arbeit. Wir haben weniger Standardcode selbst geschrieben und dafür mehr Zeit in Planung, Review und Tests gesteckt. Genau diese Verschiebung ist für uns die eigentliche Erfahrung aus dem Projekt.